

Q: What's the difference between JPA, Hibernate, and Spring Data JPA?

JPA (Java Persistence API)

JPA is a specification, not an implementation. It defines a standard set of rules and interfaces for object–relational mapping in Java, such as how entities are mapped, how queries are written, and how persistence is managed. JPA itself does not perform any database operations; it only tells how persistence should work, ensuring portability across different ORM providers.

Hibernate

Hibernate is a framework and ORM implementation that actually performs the persistence work. It implements the JPA specification and also provides additional features like advanced caching, Hibernate-specific query language (HQL), and better performance tuning options. In simple terms, Hibernate is the engine that follows JPA rules and talks to the database.

Spring Data JPA

Spring Data JPA is a Spring abstraction layer built on top of JPA that simplifies database access even further. It reduces boilerplate code by providing ready-made repository interfaces, automatic query generation, and pagination support. Spring Data JPA usually uses Hibernate internally as the JPA provider, letting developers focus more on business logic rather than persistence code.

JpaRepository in Spring Data JPA

What is JpaRepository?

JpaRepository is a Spring Data JPA interface that provides ready-made CRUD, pagination, and sorting operations for JPA entities.

By extending JpaRepository, developers don't need to write boilerplate code for basic database operations—Spring Data automatically implements it at runtime.

Key point:

JpaRepository simplifies database access and improves productivity in Spring Boot applications.

Parameters of JpaRepository

JpaRepository has two generic parameters:

1. Entity Type (T)

- The JPA entity class the repository manages.
- Represents a database table annotated with @Entity.

- Example: User, Order

2. ID Type (ID)

- The data type of the entity's primary key.
- Common types: Long, Integer, UUID, String.

One-liner explanation:

JpaRepository<Entity, ID> tells Spring Data which entity to manage and what type its primary key is.

Why use JpaRepository instead of CrudRepository?

JpaRepository provides everything CrudRepository offers, plus:

- Pagination and sorting support
- Batch operations
- Flush control
- Better JPA integration

Interview tip:

Always prefer JpaRepository in real Spring Boot projects for complete JPA functionality.

Q: When to use @Query vs derived query methods?

Derived Query Methods

Use derived query methods when the query logic is simple and clear. Spring Data JPA automatically creates the query from the method name, which makes the code clean, readable, and easy to maintain. They are best for basic conditions like equality, simple filters, and straightforward sorting.

@Query Annotation

Use @Query when the query is complex or cannot be expressed clearly using method names. It is useful for joins, custom conditions, projections, and performance-optimized queries. @Query gives you full control over the query logic.

Interview-ready summary

Use derived queries for simple CRUD and filters.

Use @Query for complex logic, joins, and performance-critical queries.

1. What is Hibernate?

Hibernate is an open-source ORM framework for mapping Java objects to relational database tables. It automates CRUD (Create, Read, Update, Delete) operations, reducing the need for manual SQL queries. Hibernate manages object persistence and database communication efficiently.

2. What are the advantages of using Hibernate

- Simplifies database operations using ORM.
- Supports HQL (Hibernate Query Language), similar to SQL but object-oriented.
- Provides caching and lazy loading for improved performance.
- Ensures database independence and portability.
- Handles transactions automatically.

3. What is ORM in Hibernate?

ORM (Object-Relational Mapping) is a technique that maps Java objects to database tables. It automates data persistence and retrieval, minimizing manual SQL handling and enabling developers to work directly with objects instead of tables.

4. Advantages of Hibernate over JDBC

Feature	Hibernate	JDBC
ORM Support	Automatic	Manual
Coding Effort	Low	High
Database Portability	High	Low
Query Caching	Supported	Not Supported
Transaction Management	Automatic	Manual

5. Important Interfaces in Hibernate

- **SessionFactory:** Creates and manages Session instances.
- **Session:** Primary interface between the application and database, maintains the first-level cache.
- **Transaction:** Represents a unit of work, handles commit and rollback.
- **Query:** Used for creating and executing HQL or native SQL queries.
- **SessionFactoryBuilder:** Builds a SessionFactory based on configuration settings.

6. List some of the databases supported by Hibernate.

MySQL, Oracle, PostgreSQL, H2, DB2, Microsoft SQL Server, Sybase, SQLite.

7. What is Java Persistence API (JPA)?

JPA is a specification for managing relational data using ORM in Java. Hibernate is one of its implementations.

- Spring Data JPA reduces boilerplate for CRUD and query operations.
- Spring Repository provides interfaces for data access with support for pagination and sorting.

8. Explain Inheritance Mapping in Hibernate

Inheritance Mapping defines how class hierarchies are stored in relational tables.

Hibernate supports:

- **Single Table:** All classes stored in one table.
- **Table per Class:** Each subclass stored in a separate table.
- **Joined Table:** Parent and child tables linked via foreign keys.

9. What is HQL?

HQL (Hibernate Query Language) is an object-oriented query language that uses entity and property names instead of table and column names. It supports aggregation, joins, and polymorphic queries.

10. How to Create HQL Queries

1. First, create an HQL Query String including entity name, property name, and relationships in the query.
2. Create a query Object by using org.hibernate.query.Query interface for creating and executing queries.
3. Also, query can be created by using createQuery()
4. Set Parameters using setParameter() is optional.
5. Execute the Query using methods like list() to retrieve the data or listing of results.
6. uniqueResult() for retrieving a single result.

```
Session session = sessionFactory.openSession();
String hql = "FROM Employee WHERE department.name = :deptName";
Query<Employee> query = session.createQuery(hql, Employee.class);
query.setParameter("deptName", "Engineering");
List<Employee> employees = query.list();
session.close();
```

11. How Can We Add Criteria to a Query?

Hibernate provides the Criteria API to build dynamic queries using an object-oriented approach.

```
Criteria criteria = session.createCriteria(Product.class);
criteria.add(Restrictions.eq("category", "Programming"));
criteria.add(Restrictions.between("price", 100.0, 500.0));
List<Product> products = criteria.list();
```

12. What is a Session in Hibernate?

Session represents the single unit of work that acts as a gateway for interacting with the databases, Hibernate session is the primary interface for working with databases.

Session provides various important functions such as

- Transaction Management
- Caching
- Lazy Loading
- Data Retrieval Strategies

- Object Persistence and Retrieval

It also handles Data Relationship Handling where developers can manage and define relationships between objects, specifying fetching strategies, and overflow of behaviors.

13. Entity Mapping Annotations (VERY COMMON).

- **@Entity**
Marks a Java class as a persistent entity that will be mapped to a database table.
- **@Table**
Specifies the table name and schema in the database for the entity. Used when the table name differs from the class name.
- **@Id**
Defines the primary key of the entity.
- **@GeneratedValue**
Specifies how the primary key value is automatically generated (e.g., AUTO, IDENTITY, SEQUENCE).
- **@Column**
Maps a class field to a database column and allows customization like nullable, unique, length, and name.
- **@Transient**
Marks a field that should not be persisted in the database (ignored by Hibernate).
- **@Enumerated**
Maps a Java enum to a database column, either as STRING or ORDINAL.
- **@Lob**
Used for storing large objects, such as large text (CLOB) or binary data (BLOB).

Interview Tip:

These annotations define how Java objects are mapped to relational database tables, enabling Hibernate to manage persistence automatically.

14. Relationship Annotation?

@OneToOne

@OneToOne represents a **one-to-one relationship** between two entities, where one record in a table is associated with exactly one record in another table. Example: User

↔ Profile. One side owns the relationship (contains the foreign key), and the other side can use mappedBy. It's commonly used for optional or extended details.

@OneToMany

@OneToMany represents a **parent-to-child relationship**, where one parent can have multiple children. Example: Order → OrderItems. This relationship is usually **mapped by the child (@ManyToOne) side** because foreign keys belong to the “many” side. Using @OneToMany without mappedBy creates an extra join table, which is often avoided.

@ManyToOne

@ManyToOne is the **most common relationship in real projects**. Many records belong to one parent. Example: many Orders belong to one Customer. This side **always owns the relationship** and contains the foreign key column in the database.

@ManyToMany

@ManyToMany represents a relationship where **many records relate to many others**. Example: Student ↔ Course. Hibernate creates a **join table** to manage the relationship. One side is marked as the owner, and the other uses mappedBy. This mapping should be used carefully due to performance and complexity.

mappedBy

mappedBy indicates the **inverse (non-owning) side** of a relationship. It tells Hibernate that the foreign key is managed by the other entity. Using mappedBy avoids duplicate join tables and ensures a single source of truth for the relationship.

Cascade Types

Cascade types define **what happens to child entities when the parent entity is acted upon**.

Common types:

- PERSIST → save child when parent is saved
- MERGE → update child when parent is updated
- REMOVE → delete child when parent is deleted

- ALL → applies all cascade operations

Cascade **does not mean database cascade**—it's handled by Hibernate at the ORM level.

Fetch Types: LAZY vs EAGER

- **LAZY (default for collections):** Data is fetched **only when accessed**. Improves performance and memory usage. Preferred in real applications.
- **EAGER (default for @ManyToOne & @OneToOne):** Data is fetched **immediately with the parent**. Can cause performance issues like N+1 queries if not handled carefully.

Best Practice: Use LAZY by default and fetch eagerly only when truly needed.

15. Difference between cascade = CascadeType.ALL and orphanRemoval = true?

cascade = CascadeType.ALL

It means: *whatever you do to the parent, Hibernate does the same to the child.*

If you **save**, **update**, or **delete** the parent, the child also gets saved, updated, or deleted.

👉 But if you **just remove the child from the parent's collection**, Hibernate **does NOT delete** it from the database.

orphanRemoval = true

It means: *if a child is removed from the parent, it is deleted from the database.*

When a child object has **no parent reference**, Hibernate treats it as an **orphan** and removes it permanently.

One-line interview answer:

- CascadeType.ALL → parent action affects child
- orphanRemoval = true → removing child from parent deletes it from DB

Golden tip (interview bonus):

Use orphanRemoval when the child **cannot exist without the parent** (like Order → OrderItem)

17. What is the N+1 Problem?

N+1 happens when Hibernate runs **1 query for parent data** and **N additional queries for child data**.

This causes serious performance issues in real projects.

In real projects, we usually use LAZY and EntityGraph to avoid N+1 issues.

19. How do you handle the N+1 query problem?

A: Use @EntityGraph, JOIN FETCH in JPQL, or batch fetching. @EntityGraph is cleanest - defines which associations to fetch in single query.

18. What is @EntityGraph?

@EntityGraph allows fetching required relationships **in a single query** without changing fetch type.

It helps solve the N+1 problem cleanly and safely.

19. What is JOIN FETCH?

JOIN FETCH tells Hibernate to **fetch related entities in the same SQL query**.

It improves performance but should be used carefully to avoid large result sets.

20. First-level Cache vs Second-level Cache

- **First-level cache:** Per session, always enabled
 - **Second-level cache:** Shared across sessions, optional
-

21. What is First-level Cache?

It is the **session-level cache**.

Hibernate automatically stores entities during a session.

It is cleared when the session ends.

22. What is Second-level Cache? It is a **shared cache across sessions**.

Used for frequently accessed and rarely changed data.

Needs external providers like Ehcache or Redis.

23. What is Query Cache?

It caches **query results**, not entities.
Must be used with second-level cache.

24. When is cache cleared?

- First-level cache → when session closes
 - Second-level cache → manually or on update
 - Query cache → when underlying data changes
-

25. Why does LazyInitializationException occur?

It occurs when a **lazy-loaded entity is accessed outside the active session/transaction**.

How to fix it:

- Use @Transactional
- Use JOIN FETCH or EntityGraph
- Fetch data inside service layer

26. What is @Transactional?

It ensures that a group of database operations execute as a single unit.
If any operation fails, all changes are rolled back.

28. Rollback Behavior

By default, Spring rolls back on runtime exceptions, not checked exceptions.

How it Works Internally (Simple Flow)

Spring uses **AOP Proxy** behind the scenes.

Flow:

Client → Proxy → Your Method

What proxy does:

1. Start transaction
2. Call your method
3. If success → commit
4. If runtime exception → rollback

You don't write commit/rollback manually.

What is Propagation?

Propagation means:

If one transactional method calls another, what should happen?

◆ **REQUIRED (default)**

- Join existing transaction
- Or create new if none exists

◆ **REQUIRES_NEW**

- Always create new transaction
- Suspend old one

👉 Used when you want separate commit (like logging).

What is Isolation?

Isolation controls:

How transactions see data from other transactions.

Common levels:

- **READ_COMMITTED** → Only committed data visible
- **REPEATABLE_READ** → Same row won't change during transaction

- **SERIALIZABLE** → Strictest, safest, slowest

It prevents issues like:

- Dirty read
 - Data changing in between
-

✓ Very Important Interview Points

- ✓ Works using **Spring AOP Proxy**
- ✓ Rolls back only on **RuntimeException** (by default)
- ✓ Doesn't work on **private methods**
- ✓ Doesn't work on **self method call inside same class**

A **private method** is a method that:

- Can be accessed **only inside the same class**
 - Cannot be called from outside the class
-

🔗 One-Line Interview Answer

"@Transactional works using Spring AOP proxy. The proxy starts a transaction before method execution and commits or rolls back based on success or exception. Propagation controls how nested transactions behave, and isolation controls data visibility between concurrent transactions."

29. Hibernate Transaction vs Spring Transaction

- Hibernate Transaction: Works at ORM level
- Spring Transaction: Manages transactions across Hibernate, JDBC, JPA

Spring Transaction is higher-level and preferred.

JPQL (Java Persistence Query Language)

What it really means:

JPQL works on Java entities and their fields, not on database tables or columns.

Hibernate converts JPQL into SQL automatically, depending on the database you use.

Why it is portable:

Since JPQL does not depend on database-specific SQL syntax, the same query works on:

- MySQL
- PostgreSQL
- Oracle
- SQL Server

When to use JPQL:

- Normal CRUD operations
- Fetching entities or DTOs
- When you want database independence

Interview line:

👉 *JPQL is preferred because it is entity-based, clean, and database-independent.*

HQL (Hibernate Query Language)

What it really means:

HQL is Hibernate's extended version of JPQL.

Everything that works in JPQL also works in HQL, but HQL adds Hibernate-only features.

Why it has more features:

- Better support for Hibernate-specific functions
- Advanced fetching options
- More flexibility in complex object mappings

Why it is less portable:

HQL is tied to Hibernate.

If you switch to another JPA provider, HQL queries may not work.

When to use HQL:

- When using Hibernate-specific features
- Complex ORM-based queries
- You are sure Hibernate will remain the provider

Interview line:

HQL is more powerful than JPQL but locks the application to Hibernate.

Native Query (SQL)

What it really means:

Native queries are pure SQL queries written exactly as the database understands them. Hibernate does not translate them — it just executes them.

Why they are best for performance tuning:

- Full control over SQL
- Can use indexes, hints, joins, window functions
- Useful for very complex queries

Why they are NOT portable:

SQL syntax changes between databases:

- LIMIT vs TOP
- Different join optimizations
- Database-specific functions

A native query written for MySQL may fail in Oracle or PostgreSQL.

When to use Native Queries:

- Very complex reporting queries
- Performance-critical operations
- Legacy databases
- Stored procedures

Interview line:

👉 *Native queries give maximum performance but reduce portability and maintainability.*

Quick Interview Comparison (One Breath Answer)

- JPQL: Entity-based, portable, preferred for most use cases
- HQL: Hibernate-specific, more powerful, less portable
- Native Query: Raw SQL, best performance, database-dependent

Final Interview Tip

If asked “Which one do you prefer?”, say:

I prefer JPQL for most cases, use HQL only when Hibernate-specific features are needed, and use native queries only for complex or performance-critical scenarios.

30. Why Pagination is Important

Fetching large data at once:

- Slows app
- Consumes memory
- Breaks UI

Pagination solves this.

31. Page vs Slice (Interview Favorite)

- Page: Gives total count (slower)
- Slice: No count query (faster)

👉 Use Slice for infinite scrolling.

32. Pageable

It bundles:

- Page number
- Page size
- Sorting

Makes pagination very easy.

33. Why Auditing is Used

Auditing helps track:

- When record was created

- When record was updated

Very important for enterprise apps.

34. Enable Auditing

Without enabling auditing globally, timestamp annotations will not work.

35. Optimistic Locking

Assumes conflicts are rare.

Uses version check before update.

Best for:

- High-read systems
 - Web applications
-

36. Pessimistic Locking

Locks DB row immediately.

Prevents other users from updating.

Best for:

- Banking
 - Critical transactions
-

37. @Version (Why Important)

Prevents lost updates in concurrent systems.

Interviewers love this topic.

38. Batch Processing

Processes large data in chunks.

Reduces DB calls and memory usage.

39. DTO Projection

Fetches only required data.

Improves performance and API clarity.

40 @Modifying Queries

Used when query changes DB state.

41. @Transactional(readOnly = true)

Optimizes DB interaction for read-only logic.

42. Soft Delete

Data is not removed physically.

Used for audit, recovery, and compliance.

42. What's the difference between save() and saveAndFlush()?

A: save() persists to persistence context, saveAndFlush() immediately writes to database. Use saveAndFlush() when you need the generated ID immediately or for batch operations.

43. How do you implement custom repository methods?

A: Create custom interface, implement it with @Repository, and extend in your main repository interface. Access EntityManager for complex queries via @PersistenceContext.

44. What's @Version used for?

A: Optimistic locking - prevents concurrent updates by checking version field. Hibernate automatically increments version on updates, throws exception on version mismatch.

46. What is Persistence Context?

Answer:

The persistence context is a **memory space** where Hibernate tracks all entities that are managed in the current session or transaction.

Any entity inside the persistence context is automatically synchronized with the database.

Why it matters:

- Prevents duplicate DB queries
- Tracks entity changes automatically
- Enables dirty checking

Interview line:

👉 Persistence context acts like a **first-level cache and change tracker**.

48 Entity States in Hibernate?**Transient**

An entity object is created but **not associated with Hibernate** and not stored in the database.

Persistent

The entity is **associated with a session** and tracked by Hibernate.
Any change is automatically synchronized with the database.

Detached

The entity was persistent earlier but the session is now closed.
Changes are **not tracked** automatically.

Removed

The entity is marked for deletion and will be removed on transaction commit.

Interview line:

👉 Hibernate manages entities through lifecycle states inside the persistence context.

50 Difference Between flush() and commit()**flush():**

Synchronizes the persistence context with the database.
SQL statements are executed but **transaction is not finalized**.

commit():

Finalizes the transaction and permanently saves changes.

Key point:

Flush can happen multiple times, commit happens once.

51. What is DTO and Mapper?

DTO (Data Transfer Object)

DTO is a **simple object used to transfer data** between layers (Controller → Service → Client).

It contains **only required fields**, no business logic.

DTO helps **hide internal entity structure**, improves **performance**, and avoids **over-fetching data**.

Interview line:

👉 DTO is used to send only necessary data outside the application.

Mapper

A Mapper is a component that **converts one object to another**, usually **Entity ↔ DTO**.

It separates conversion logic from business logic, making code **clean and maintainable**.

Interview line:

👉 Mapper handles transformation between Entity and DTO.

50) What is DAO?

DAO (Data Access Object)

DAO is a design pattern that **handles all database operations**.

It provides methods for **CRUD operations** and **hides database logic** from the business layer.

Why DAO is used:

- Separates persistence logic from business logic
- Makes code easier to maintain and test
- Improves modularity

Interview line:

👉 DAO is responsible only for database access.

51) @CreationTimestamp & @UpdateTimestamp

- **@CreationTimestamp** → Automatically sets the **creation time** when the entity is first saved.
- **@UpdateTimestamp** → Automatically updates the **modification time** whenever the entity is updated.
- **On deletion:**
 - **Physical delete:** record removed, timestamps lost
 - **Soft delete:** update a flag and @UpdateTimestamp can track deletion time